

Memory defenses (II)

Mainack Mondal

CS60112

Spring 2023-24



The road ahead

- Memory attacks
 - Buffer overflow
 - Ret2libc
 - Integer overflow
 - Format string vulnerabilities
 - Heap attacks
- Defenses
 - DEP
 - Stack canaries
 - ASLR
 - Reference monitor: Enforceable security policies

But before that...

A bit of news: tar xz backdoor attack...

From media, with scare



SUPPLY CHAIN ATTACK —

Backdoor found in widely used Linux utility targets encrypted SSH connections

Malicious code planted in xz Utils has been circulating for more than a month.

DAN GOODIN - 3/30/2024, 12:20 AM

SECURITY ALERT: POTENTIAL SSH BACKDOOR VIA LIBLZMA

by: Jonathan Bennett

34 Comments



March 29, 2024



So, what happened?

- The detection



Products	Services	Publications	Resources	What's new
----------	----------	--------------	-----------	------------

[Follow @Openwall on Twitter for new release announcements and other news](#)
[\[<prev\]](#) [\[next>\]](#) [\[thread-next>\]](#) [\[day\]](#) [\[month\]](#) [\[year\]](#) [\[list\]](#)

Date: Fri, 29 Mar 2024 08:51:26 -0700
From: Andres Freund <andres@...razel.de>
To: oss-security@...ts.openwall.com
Subject: backdoor in upstream xz/liblzma leading to ssh server compromise

Hi,

After observing a few odd symptoms around liblzma (part of the xz package) on Debian sid installations over the last weeks (logins with ssh taking a lot of CPU, valgrind errors) I figured out the answer:

The upstream xz repository and the xz tarballs have been backdoored.

At first I thought this was a compromise of debian's package, but it turns out to be upstream.

== Compromised Release Tarball ==

One portion of the backdoor is **solely* in the distributed tarballs*. For easier reference, here's a link to debian's import of the tarball, but it is also present in the tarballs for 5.6.0 and 5.6.1:

<https://www.openwall.com/lists/oss-security/2024/03/29/4>

So, what happened?

- From MS, seriously? Yes.



AndresFreundTec

@AndresFreundTec@mastodon.social

I was doing some micro-benchmarking at the time, needed to quiesce the system to reduce noise. Saw sshd processes were using a surprising amount of CPU, despite immediately failing because of wrong usernames etc. Profiled sshd, showing lots of cpu time in liblzma, with perf unable to attribute it to a symbol. Got suspicious. Recalled that I had seen an odd valgrind complaint in automated testing of postgres, a few weeks earlier, after package updates.

Really required a lot of coincidences.

So, what happened?

- From MS, seriously? Yes.



AndresFreundTec

@AndresFreundTec@mastodon.social

I was doing some micro-benchmarking at the time, needed to

quiesce the

using a su

because o

cpu time i

suspicious

automate

updates.

Really req

With the backdoored liblzma installed, logins via ssh become a lot slower.

```
time ssh nonexistent@...alhost
```

before:

```
nonexistent@...alhost: Permission denied (publickey).
```

before:

```
real    0m0.299s
user    0m0.202s
sys     0m0.006s
```

after:

```
nonexistent@...alhost: Permission denied (publickey).
```

```
real    0m0.807s
user    0m0.202s
sys     0m0.006s
```

And the memes started...



vx-underground  @vxunderground · Mar 30



The xz backdoor was initially caught by a software engineer at Microsoft. He noticed **500ms** lag and thought something was suspicious.

This is the Silver Back Gorilla of nerds. The internet final boss.

And the memes started...



vx-underground ✓ @vxunderground · Mar 30



The xz backdoor was initially caught by a software engineer at Microsoft.

He noticed 500ms lag and thought something was suspicious.



sophie ✓ @netcapgirl · Mar 31



hey bro, not to be weird or anything but you seem to be taking **500ms** longer than usual to do stuff, you sure you're good?

💬 24

↻ 26

♥ 408

📊 40K



And the memes started...



vx-underground ✓ @vxunderground · Mar 30



The xz backdoor was initially caught by a software engineer at Microsoft.

He noticed 500ms lag and thought something was suspicious.



sophie ✓ @netcapgirl · Mar 31



hey bro, not to be weird or anything but you seem to be taking **500ms** longer than usual to do stuff, you sure you're good?



vx-underground ✓ @vxunderground · Mar 30



Microsoft engineer: **500ms** lag in liblzma? Something's up.

Also Microsoft engineer: 45 minute lag in Microsoft Teams? Perfect.

💬 133

↻ 2.3K

❤ 22K

📊 867K



And the memes started...



vx-underground ✓ @vxunderground · Mar 30

The xz backdoor was initially caught by a software engineer at Microsoft.

He noticed 500ms lag and thought something was suspicious.



sophie ✓ @netcapgirl · Mar 31

hey bro, not to be weird or anything but you seem to be taking **500ms** longer than usual to do stuff, you sure you're good?



vx-underground ✓ @vxunderground · Mar 30

Microsoft engineer: **500ms** lag in liblzma? Something's up.



Mike Coutermarsh ✓ @mscccc · Mar 31

Notices **500ms** latency

MSFT Engineer: solves problem, sends text only email. Saves world.

You: Blames framework, declares Vercel is dead, 12 tweets a day. Still \$0 MRR.



11



60



1.1K



111K



And the memes started...



vx-underground ✓ @vxunderground · Mar 30

The xz backdoor was initially caught by a software engineer at Microsoft.

He noticed 500ms lag and thought something was suspicious.



sophie ✓ @netcapgirl · Mar 31

hey bro, not to be weird or anything but you seem to be taking **500ms** longer than usual to do stuff, you sure you're good?



vx-underground ✓ @vxunderground · Mar 30

Microsoft engineer: **500ms** lag in liblzma? Something's up.



Mike Coutermarsh ✓ @mscccc · Mar 31

Notices **500ms** latency

MSFT Engineer: solves problem, sends text only email. Saves world.



Himiko Hazz 卩 (hazzim.bsky.social) @HazzimIO · Apr 1

You know the guy who discovered the backdoor in XZ is old school because he wants to find the reason for the **500 ms** latency and fix it, instead of increasing the RAM or CPU.



1



7



62



2K



lay. Still \$0



Two parts of the attack

1. Social engineering
2. Creating the attack

Social engineering

Over a period of over two years, an attacker using the name “Jia Tan” worked as a diligent, effective contributor to the xz compression library, eventually being granted commit access and maintainership.

Using that access, they installed a very subtle, carefully hidden backdoor into liblzma, a part of xz that also happens to be a dependency of OpenSSH sshd on Debian, Ubuntu, Fedora, and other systemd-based Linux systems. That backdoor watches for the attacker sending hidden commands at the start of an SSH session, giving the attacker the ability to run an arbitrary command on the target system without logging in: unauthenticated, targeted remote code execution. The attack appears to be the first serious known supply chain attack on widely used open source software. It marks a watershed moment in open source supply chain security, for better or worse.

- <https://research.swtch.com/xz-timeline>

Social engineering

Prologue

2005–2008: [Lasse Collin, with help from others](#), designs the .xz file format using the LZMA compression algorithm, which compresses files to about 70% of what gzip did [1]. Over time this format becomes widely used for compressing tar files, Linux kernel images, and many other uses.

Social engineering

Jia Tan arrives on scene, with supporting cast

2021-10-29: Jia Tan sends [first, innocuous patch](#) to the xz-devel mailing list, adding “.editorconfig” file.

2021-11-29: Jia Tan sends [second innocuous patch](#) to the xz-devel mailing list, fixing an apparent reproducible build problem. More patches that seem (even in retrospect) to be fine follow.

2022-04-19: Jia Tan sends [yet another innocuous patch](#) to the xz-devel mailing list.

2022-04-22: “Jigar Kumar” sends [first of a few emails](#) complaining about Jia Tan’s patch not landing. (“Patches spend years on this mailing list. There is no reason to think anything is coming soon.”) At this point, Lasse Collin has already landed four of Jia Tan’s patches, marked by “Thanks to Jia Tan” in the commit message.

2022-05-19: “Dennis Ens” sends [mail to xz-devel](#) asking if XZ for Java is maintained.

Social engineering

Jia Tan arrives on scene, with supporting cast

2021-10-29: Jia Tan sends [first, innocuous patch](#) to the xz-devel mailing list, adding “.editorconfig” file.

2021-11-29: Jia Tan sends [second innocuous patch](#) to the xz-devel mailing list, fixing an apparent reproducible build problem. More patches that seem (even in retrospect) to be fine follow.

2022-04-19: Jia Tan sends [yet another innocuous patch](#) to the xz-devel mailing list.

2022-04-22: “Jigar Kumar” sends [first of a few emails](#) complaining about Jia Tan’s patch not landing. (“Patches spend years on this mailing list. There is no reason to think anything is coming soon.”) At this point, Lasse Collin has already landed four of Jia Tan’s patches, marked by “Thanks to Jia Tan” in the commit message.

2022-06-14: Jugar Kumar sends [pressure email](#). “With your current rate, I very doubt to see 5.4.0 release this year. The only progress since april has been small changes to test code. You ignore the many patches bit rotting away on this mailing list. Right now you choke your repo. Why wait until 5.4.0 to change maintainer? Why delay what your repo needs?”

2022-06-21: Dennis Ens sends [pressure email](#). “I am sorry about your mental health issues, but its important to be aware of your own limits. I get that this is a hobby project for all contributors, but the community desires more. Why not pass on maintainership for XZ for C so you can give XZ for Java more attention? Or pass on XZ for Java to someone else to focus on XZ for C? Trying to maintain both means that neither are maintained well.”

2022-06-22: Jigar Kumar sends [pressure email](#) to C patch thread. “Is there any progress on this? Jia I see you have recent commits. Why can't you commit this yourself?”

Social engineering

- **2022-06-29:** Lasse Collin replies: “As I have hinted in earlier emails, Jia Tan may have a bigger role in the project in the future. He has been helping a lot off-list and is practically a co-maintainer already. :-) I know that not much has happened in the git repository yet but things happen in small steps. In any case some change in maintainership is already in progress at least for XZ Utils.”
- **Jia Tan becomes maintainer**
- At this point Lasse seems to have started working even more closely with Jia Tan. Evan Boehs observes that Jigar Kumar and Dennis Ens both had nameNNN@mailhost email addresses that never appeared elsewhere on the internet, nor again in xz-devel. It seems likely that they were fakes created to push Lasse to give Jia more control. It worked. Over the next few months, Jia started replying to threads on xz-devel authoritatively about the upcoming 5.4.0 release.

Two parts of the attack

1. Social engineering

- Gained maintainer powers to a repository used by most all linux machines
- <https://research.swtch.com/xz-timeline>
- I also have a book for you:
[https://en.wikipedia.org/wiki/The_Cuckoo%27s_Egg_\(book\)](https://en.wikipedia.org/wiki/The_Cuckoo%27s_Egg_(book))

2. Creating the attack

Two parts of the attack

1. Social engineering

- Gained maintainer powers to a repository used by most all linux machines
- <https://research.swtch.com/xz-timeline>
- I also have a book for you:
[https://en.wikipedia.org/wiki/The_Cuckoo%27s_Egg_\(book\)](https://en.wikipedia.org/wiki/The_Cuckoo%27s_Egg_(book))

2. Creating the attack

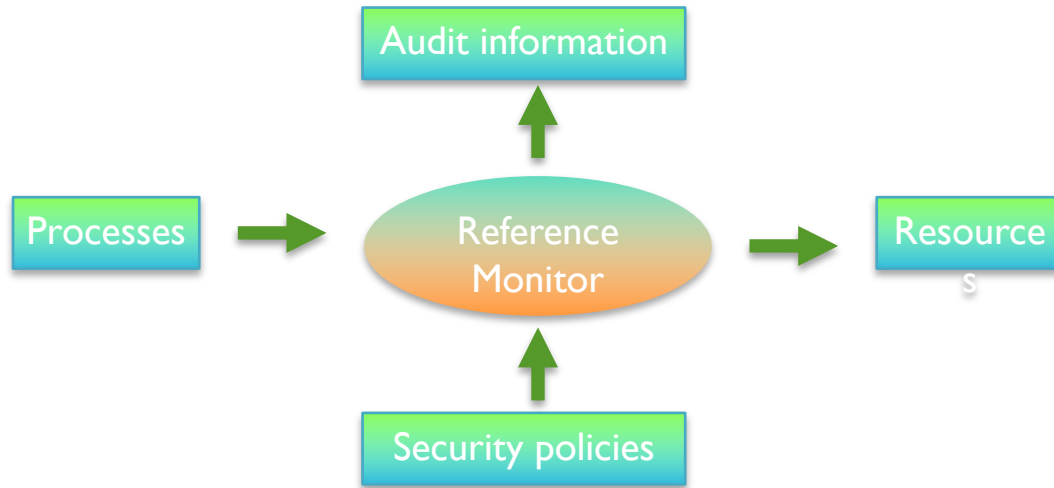
- <https://gist.github.com/thesamesam/223949d5a074ebc3dce9ee78baad9e27>
- <https://gynvael.coldwind.pl/?lang=en&id=782>

Parts of the attack is still not understood...

Any takers?

Anywho, lets go back

Reference monitors



- Observes execution of the program/process
- First question: **at what level?** Possibilities: hardware, OS, network
 - The level determines what the monitor can see
- Answer depends on the second question: **which “security policy”?**
 - Which system events are relevant to the policy? what do we have to monitor?
 - Instructions, memory accesses, system calls, network packets...

Enforceable security policies

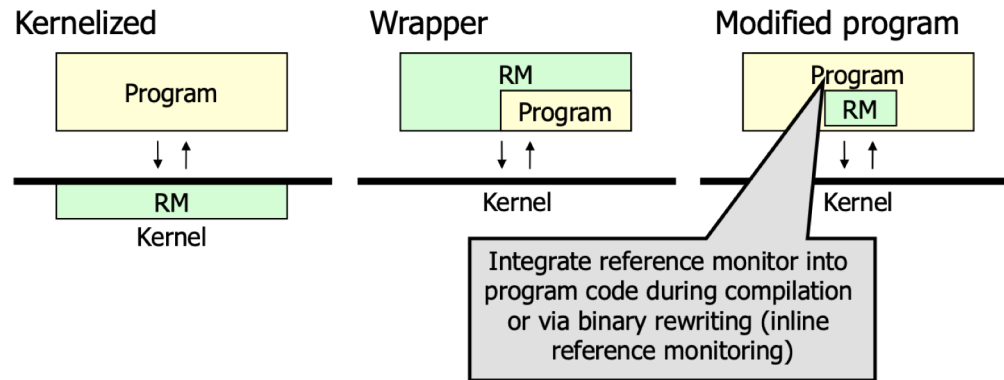
Enforceable Security Policies

FRED B. SCHNEIDER
Cornell University

ACM TISSEC '98

- Reference monitors can only enforce **safety policies**
 - Execution of a process is a sequence of states
 - Safety policy is a predicate on a prefix of the sequence
 - Policy must depend only on the past of a particular (i.e., the currently monitored) execution; once it becomes false, it's always false
- Not policies that require knowledge of the future
 - “If this server accepts a SYN packet, it will eventually send a response”

Reference monitor implementation



- **Kernelised:**
 - high protection, requires changes in the OS, operates on OS calls
- **Wrapper:**
 - less protection but does not require changes in the OS
- **Inline reference monitoring:**
 - the program might try to tamper with the monitor (e.g., JavaScript)
 - does not require changes in the OS
 - policies can depend on application semantics (e.g., function calls)
 - no context switches, hence lower performance overhead

What makes a process safe?

- **Memory safety:** all memory accesses are “correct”
 - Respect array bounds, don’t stomp on another process’s memory, don’t execute data as if it were code
- **Control-flow safety:**
 - all control transfers are envisioned by the original program
 - No arbitrary jumps, no calls to library routines that the original program did not call
- **Type safety:**
 - all function calls and operations have arguments of correct type

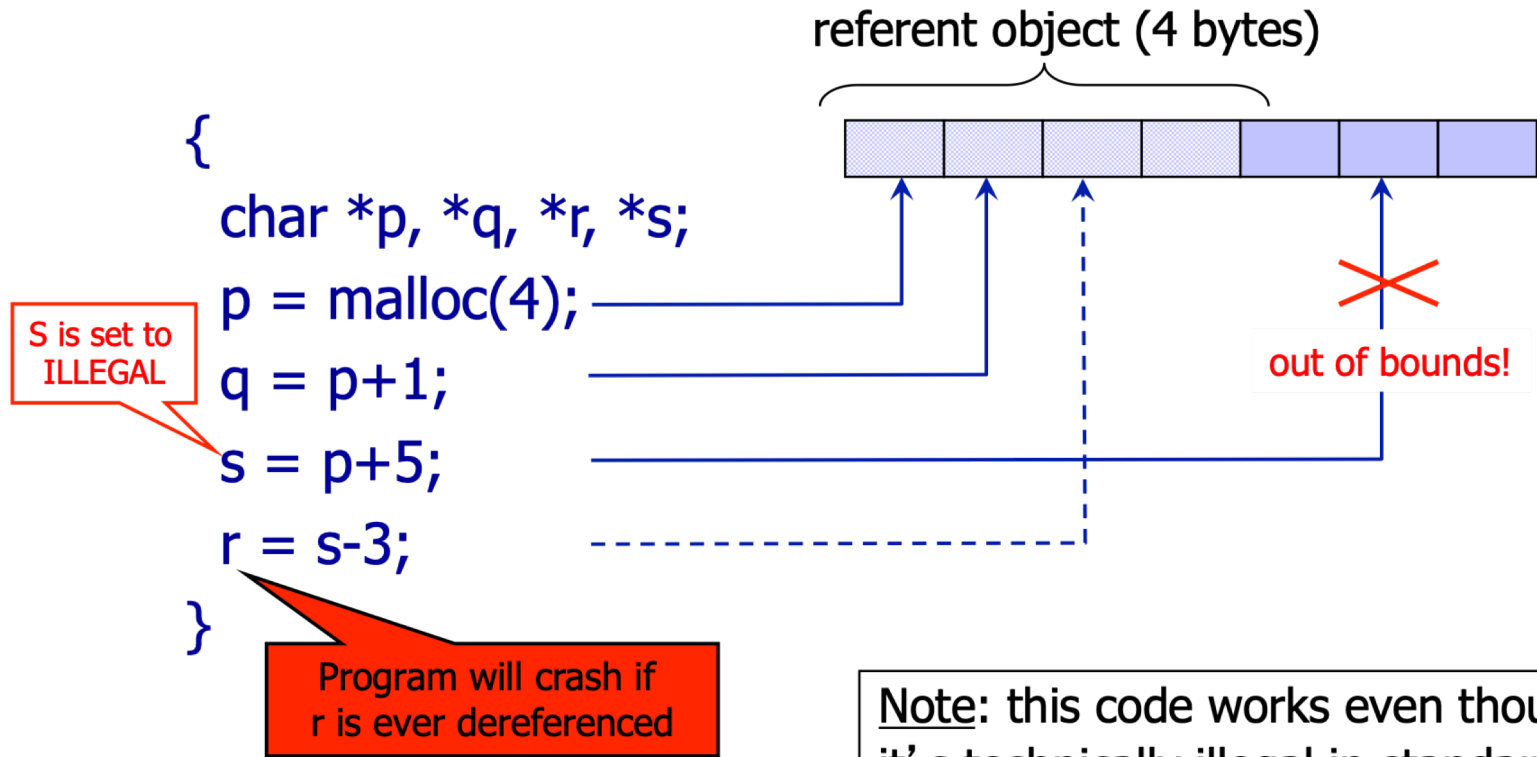
Runtime bounds checking

- Problem: **avoid OOB accesses** to the **intended referent** (intended target of each pointer)
- Observation: **actual size is available at runtime!**
- Three approaches:
 - Use an expanded pointer representation (“fat-pointers”) to record information about the intended referent to each object
 - Incompatible with external code, libraries, etc. 😞
 - Store the metadata separately from the pointer but use a map (e.g., a hash table) from pointers to metadata
 - still incompatibility issues: pointers possibly modified by an external library must have their metadata updated at a library call
 - potential high costs due to map searches on each dereference 😞
 - Store only the address ranges of live objects and ensure that pointer arithmetic never crosses out of the original object into another valid object
 - For every pointer arithmetic operation, check that the result points to the same referent object
 - backwards compatible!

Jones-Kelly

- Maintain a runtime splay tree of allocated objects storing a description thereof
 - base, extent, additional information
- Check in the table for each pointer arithmetic operation
 - requires a tree traversal
- Backwards-compatible pointer representation
- Replace all out-of-bounds addresses with special ILLEGAL value (if dereferenced, program crashes)
- Problem: what if a pointer to an out-of-bounds address is used to compute an in-bounds address?
- Result: false alarm

Example of a false alarm



Ruwase-Lam

Olatunji Ruwase

Monica S. Lam

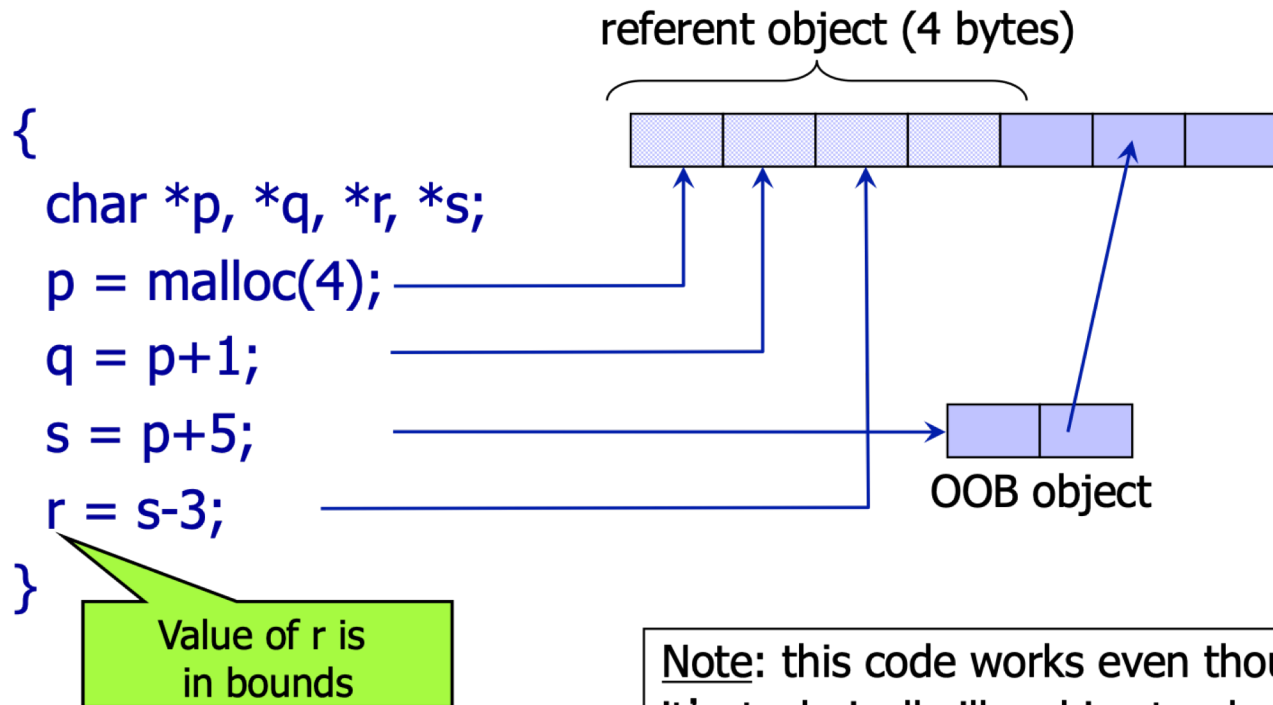
Transmeta Corporation
3990 Freedom Circle
Santa Clara, CA 95054
tjruwase@transmeta.com

Computer Systems Laboratory
Stanford University
Stanford, CA 94305
lam@stanford.edu

A tilted rectangular logo with a green-to-orange gradient background. The text "NDSS'04" is written in bold black font, rotated slightly counter-clockwise.

- Instead of ILLEGAL, make each out-of-bounds (OOB) pointer point to a special **OOB object**
 - Stores the original out-of-bounds value
 - Stores a pointer to the original referent object
- Pointer arithmetic on out-of-bounds pointers
 - Simply use the actual value stored in the OOB object
- If a pointer is dereferenced, check if it points to an actual object. If not, halt the program!

Example of an OOB object



Note: this code works even though it's technically illegal in standard C

Performance issues

- Checking the referent object table on every pointer arithmetic operation is very expensive
- Jones-Kelly: 5x-6x slowdown
 - Tree of allocated objects grows very big
- Ruwase-Lam: 11x-12x slowdown if enforcing bounds on all objects, up to 2x if only strings
- Unusable in production code!

Dhurjati-Arve

- Split memory into disjoint pools
 - static pointer analysis to merge all the target objects of a pointer into a single pool
 - target pool for each pointer known at compile-time
- Separate tree of allocated objects for each pool
 - Smaller tree, much faster lookup; also caching
- Instead of returning a pointer to an OOB, return an address from the kernel address space
 - Separate table maps this address to the OOB
 - Don't need checks on every dereference: OOB memory accesses will cause a hardware trap!

OOB pointers: Ruwase-Lam

```
p = malloc(10 * sizeof(int));
```

```
q = p + 20;
```

```
r = q - 15;
```

```
*r = ... ; //no bounds overflow
```

```
*q = ... ; // overflow
```

```
q = OOB(p+20,p)
```

Put OOB(p+20,p) into a map

```
r = p + 5
```

Check if r is out of bounds

Check if q is out of bounds:

Runtime error

Check on every dereference

OOB pointers: Dhurjati-Arve

```
p = malloc(10 * sizeof(int));  
q = p + 20;
```

```
r = q - 15;
```

```
*r = ... ; //no bounds overflow
```

```
*q = ... ; // overflow
```

```
q = 0xCCCCCCCC  
Put (0xCCCCCCCC, OOB(p+20,p))  
into a map
```

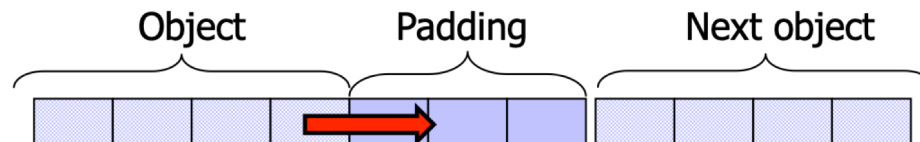
```
r = p + 5
```

No software check necessary!

No software check necessary!
Runtime error

Baggy bounds

- Allocators pad objects to align pointers
- Insight: to prevent “bad” out-of-bounds memory accesses, it is enough to check allocation bounds, not the precise object bounds



Out-of-bounds access to padding is harmless

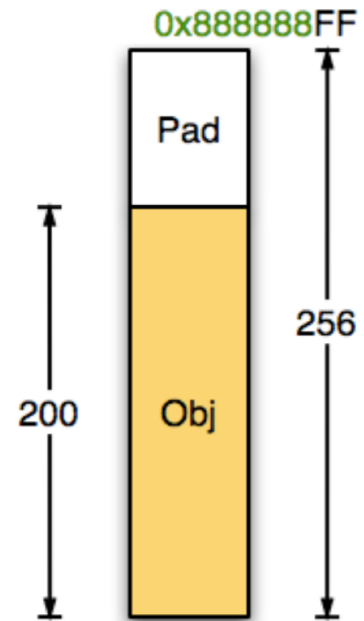
Very efficient bounds representation

- Storing the pointer to the object and its size requires at least 8 bytes per object
- Instead, use a custom allocator to pad and align objects to powers of 2
 - it's enough to store logarithm of object's size in the bounds table

$$e = \log_2(\text{alloc_size})$$

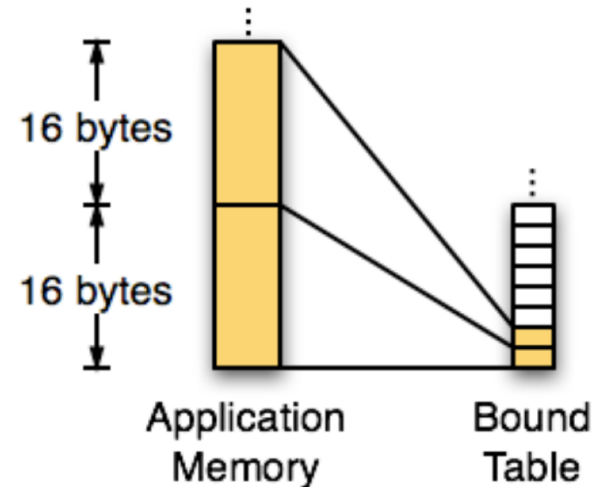
- This takes 1 byte per object
- It can be used to compute its size and base pointer:

```
alloc_size = 1 << e  
base = p & ~(alloc_size-1)
```



Very efficient bounds table

- Partition memory into slots and align allocated objects to slot boundaries
 - Thus each slot can belong to at most 1 object
- Bounds table = contiguous array of 1-byte entries (an entry per each slot)
- Given an address p , finding its entry takes a **single memory lookup**
 - $p \gg \log_2(\text{slot_size}) + \text{constant table base}$
= address of the corresponding entry in the bounds table
 - No need for tree traversal!



Benign OOB accesses

- OOB accesses within the padding are allowed
- Since each slot contains at most one object, they cannot be exploited for typical attacks such as
 - overwriting a return address, function pointer or other security critical data
 - or reading sensitive information from another object, such as a password

Very efficient bounds checking

- Given a pointer arithmetic operation ...

$$p' = p + i$$

- ... perform a very efficient check

$$(p \oplus p') \gg \text{BoundsTable}[p \gg \log_2(\text{slot_size})] == 0$$

- This checks whether p and p' have the same prefix with only $\log(\text{alloc_size})$ least significant bytes modified
- No need to check against lower and upper bounds!

Handling out-of-bound pointers

- Use a variant of Dhurjati-Adve technique to prevent OOB pointers from being dereferenced
 - Restrict the program to lower half of address space, set the most significant bit of OOB pointers to 1, thus all OOB look as if they point into kernel space
- Find the original referent object by checking whether the OOB pointer is in the top or bottom half of a slot
 - Only works within $\text{slot_size}/2$ of the original object
 - On 64-bit architectures, can do better by using “spare” bits to tag each pointer with its bounds info

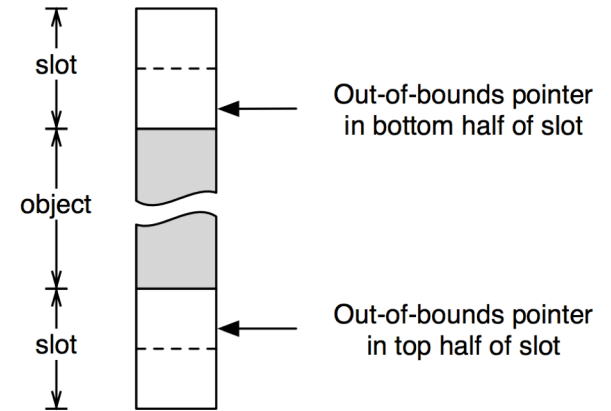


Figure 4: We can tell whether a pointer that is out-of-bounds by less than $\text{slot_size}/2$ is below or above an allocation. This lets us correctly adjust it to get a pointer to the object by respectively adding or subtracting slot_size .

Required reading

<https://github.com/alinush/6.858-lecture-notes/blob/master/I02-baggy.md>

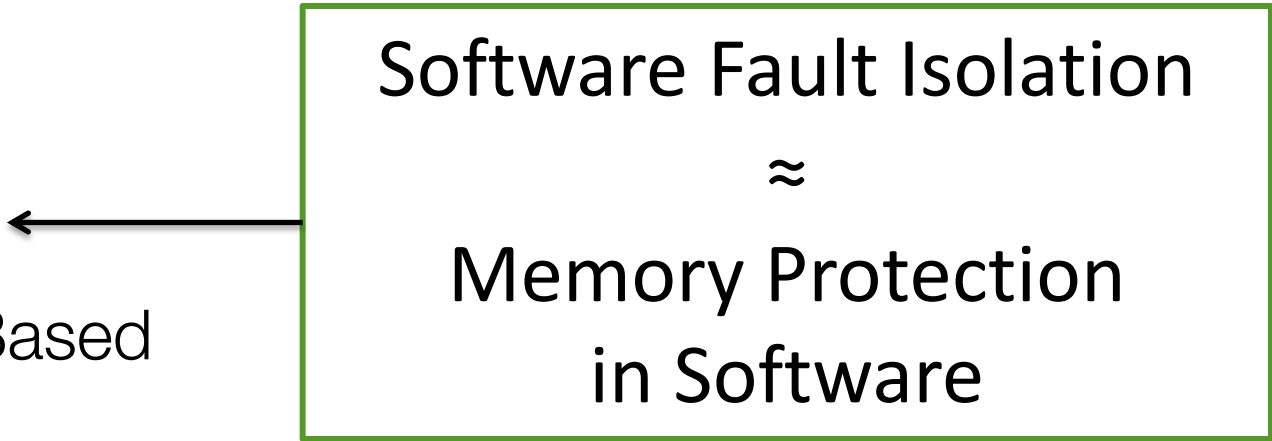
Fault isolation in reference monitors

(slide courtesy David Brumley, CMU)

Isolation Mechanisms

- Hardware
 - Memory Protection (virtual address translation, x86 segmentation)
- Software
 - Sandboxing
 - Language-Based
- Hardware + Software
 - Virtual machines

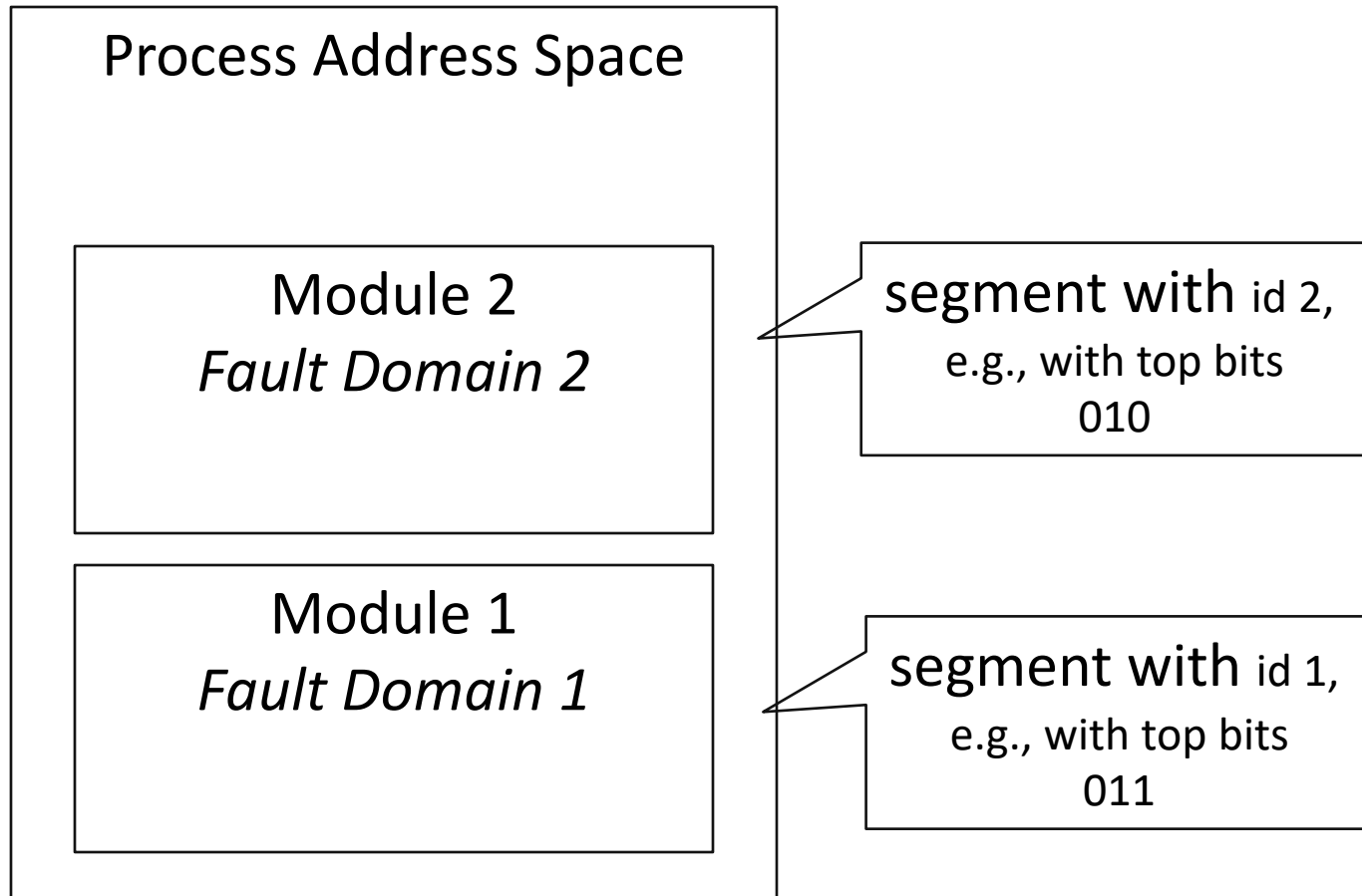
Software Fault Isolation
≈
Memory Protection
in Software



SFI Goals

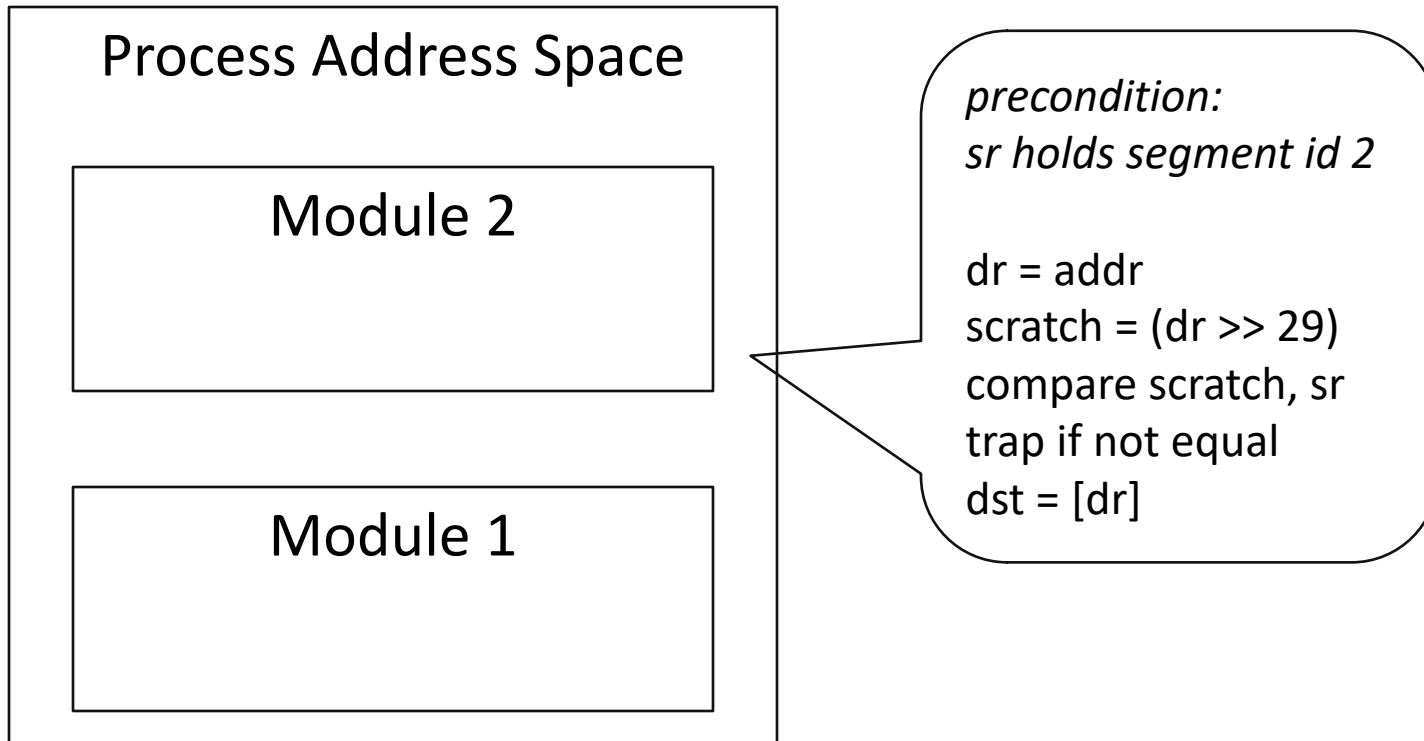
- Confine faults inside distrusted extensions
 - codec shouldn't compromise media player
 - device driver shouldn't compromise kernel
 - plugin shouldn't compromise web browser
- Allow for efficient cross-domain calls
 - numerous calls between media player and codec
 - numerous calls between device driver and kernel

Main Idea



Scheme 1: Segment Matching

- Check every mem access for matching seg id
- assume dedicated registers segment register (sr) and data register (dr)
 - not available to the program (no big deal in Alpha)

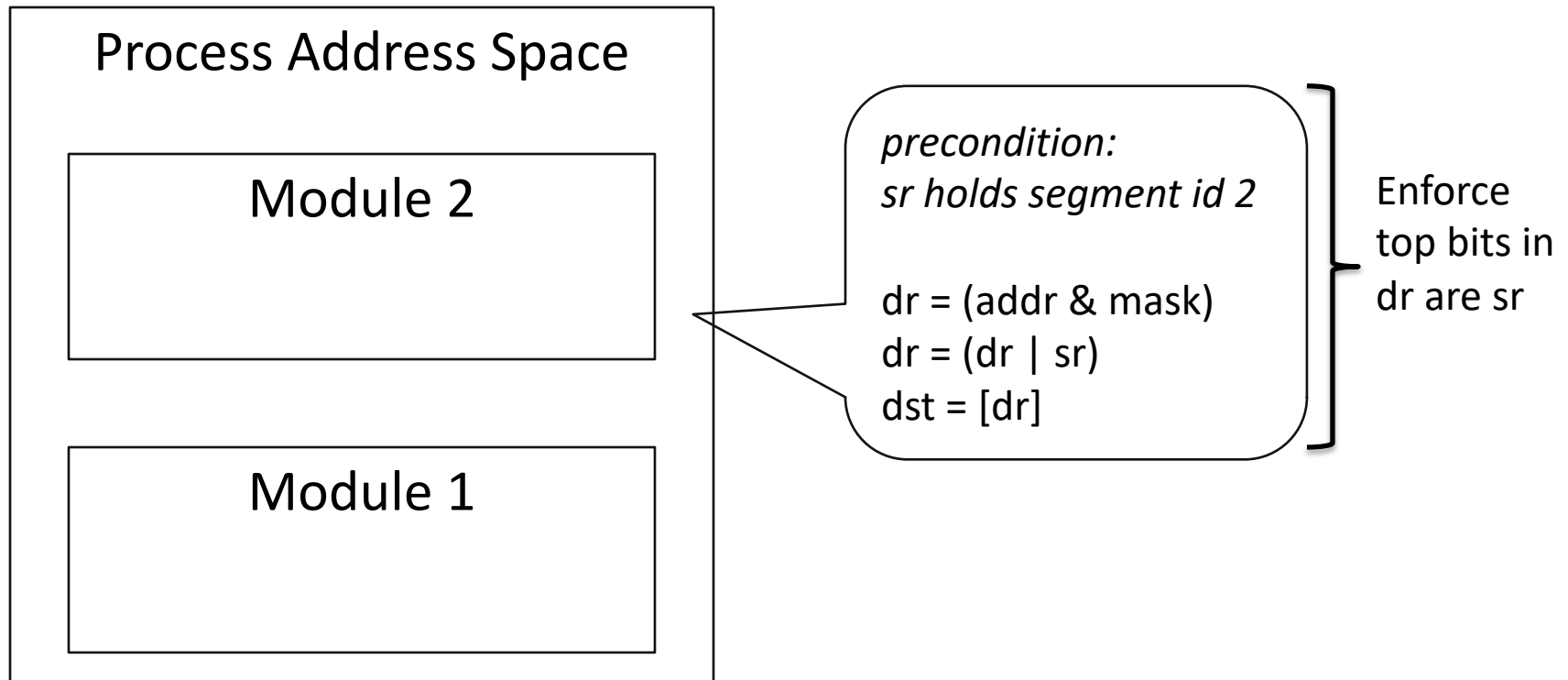


Safety

- Segment matching code must always be run to ensure safety.
- Dedicated registers must not be writeable by module.

Scheme 2: Sandboxing

- Force top bits to match seg id and continue
- No comparison is made



Segment Matching vs. Sandboxing

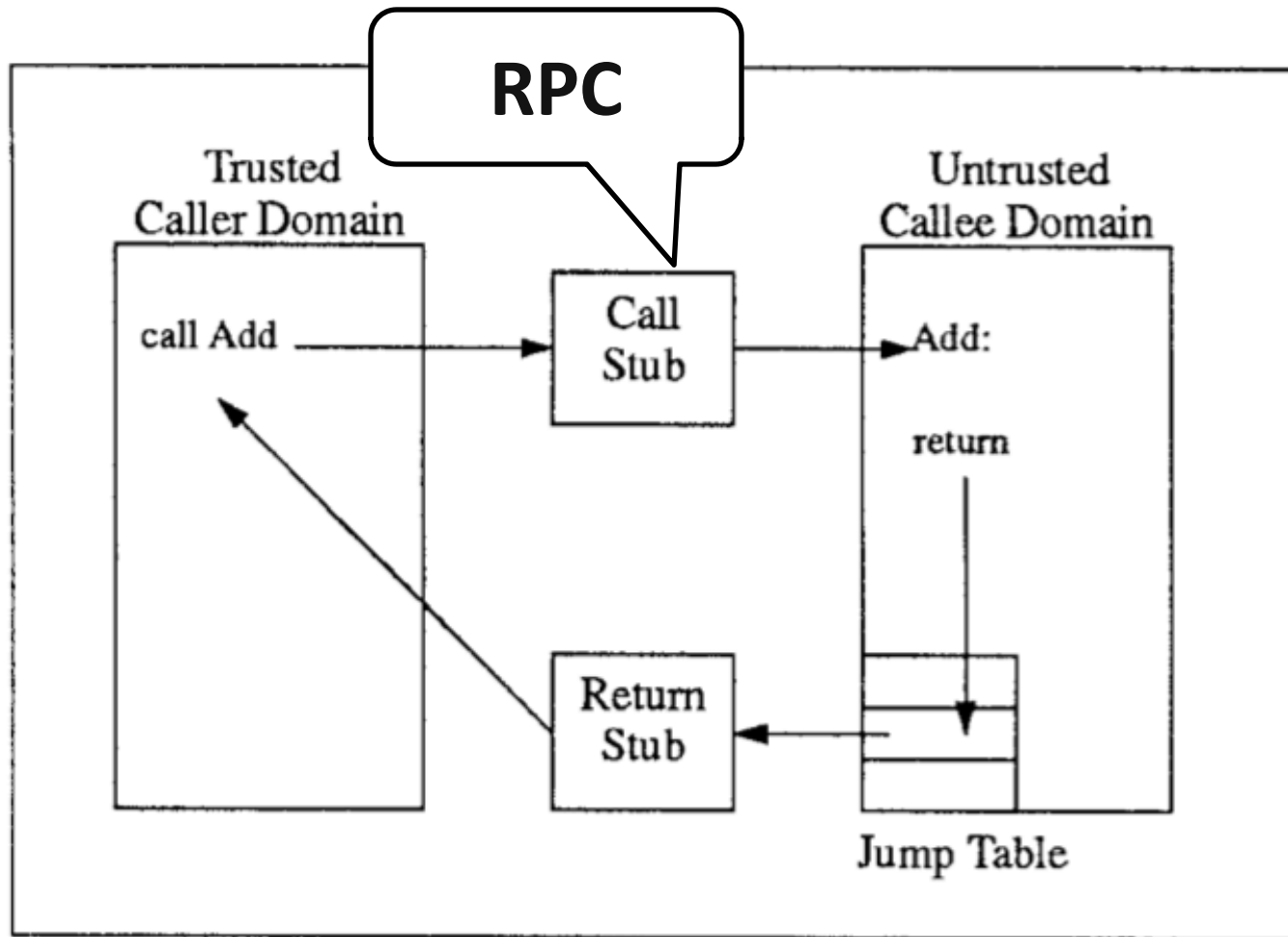
Segment Matching

- more instructions
- can pinpoint exact point of fault where segment id doesn't match

Sandboxing

- fewer instructions
- just ensures memory access stays in region (crash is ok)

Communication between domains



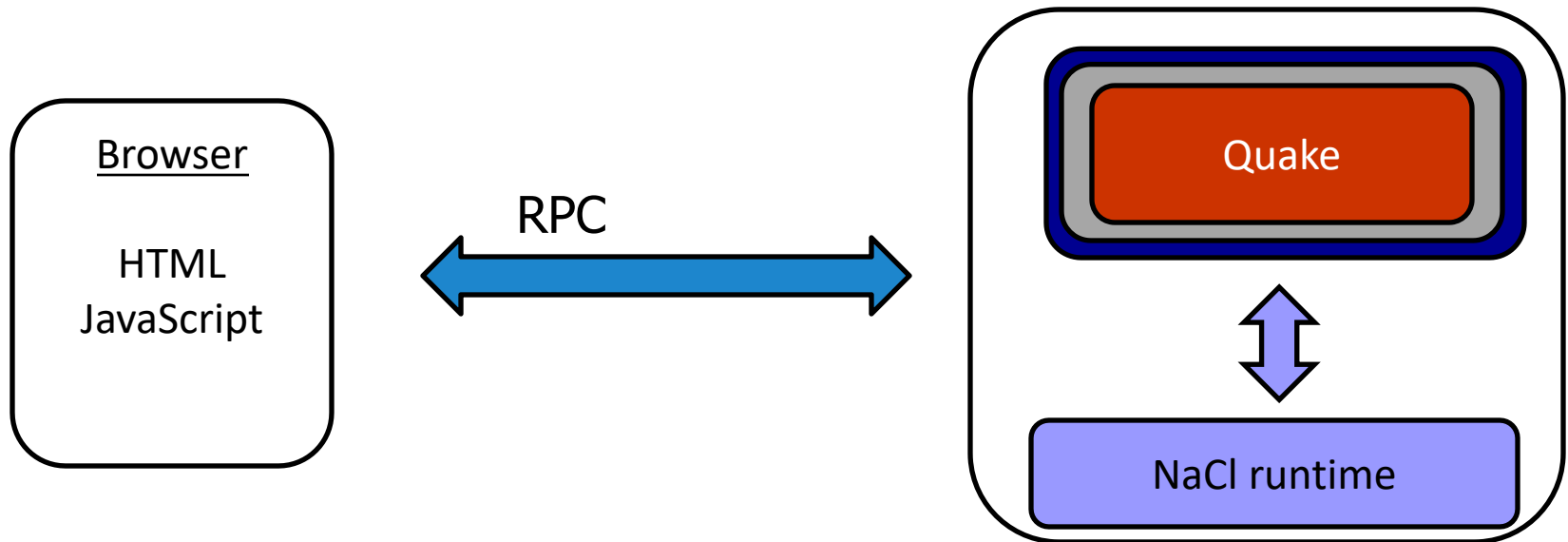
NATIVE CLIENT

Optional Reading:

*Native Client: A Sandbox for Portable,
Untrusted x86 Native Code*

by Yee et al.

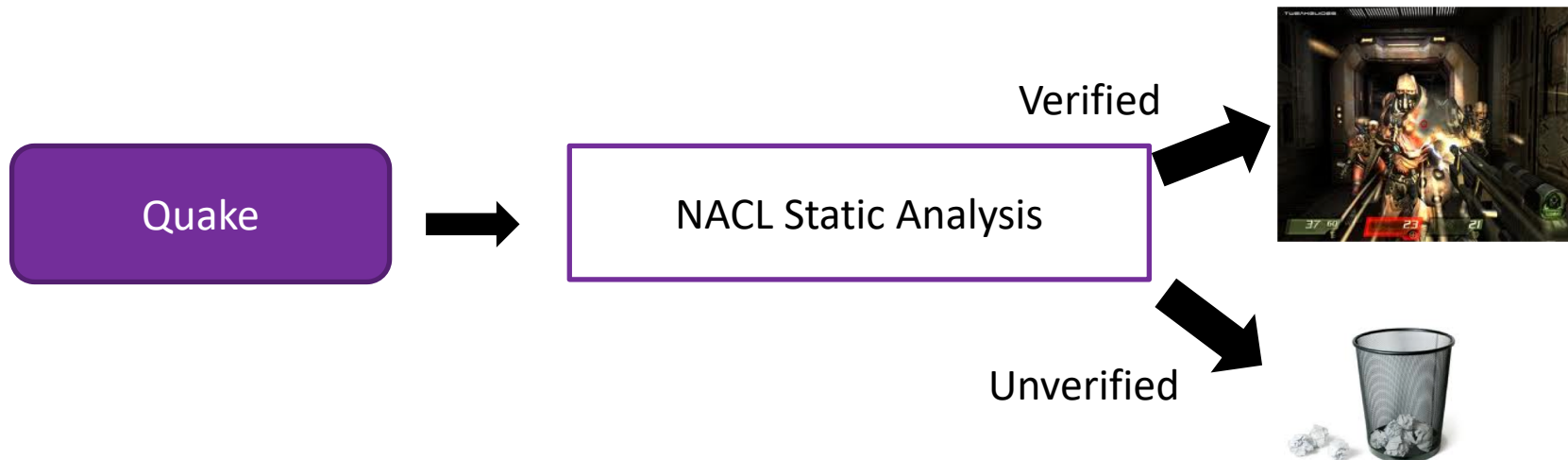
NaCl: A Modern Day Example



- Two sandboxes:
 - an inner sandbox to mediate x86-specific runtime details
 - an outer sandbox mediates system calls

Security Goal

- Achieve comparable safety to accepted systems such as JavaScript.
 - Input: **arbitrary** code and data
 - support multi-threading, inter-module communication
 - NaCL checks that code conforms to security rules, else refuses to run.



Obligations

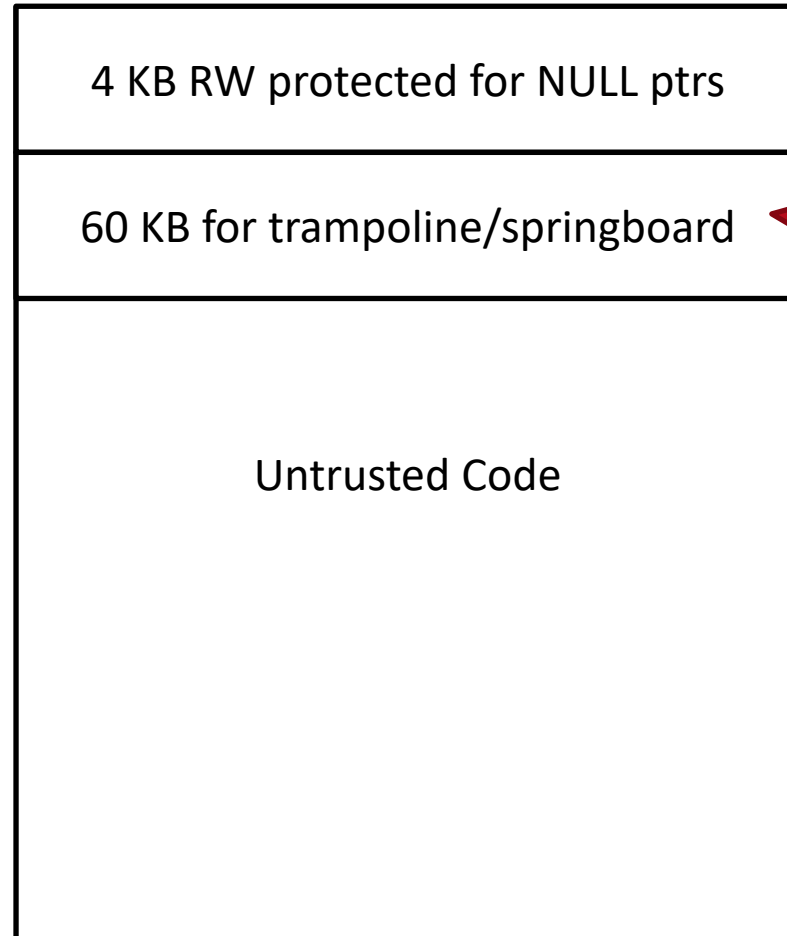
- | | |
|----|--|
| C1 | Once loaded into the memory, the binary is not writable, enforced by OS-level protection mechanisms during execution. |
| C2 | The binary is statically linked at a start address of zero, with the first byte of text at 64K. |
| C3 | All indirect control transfers use a <code>nacljmp</code> pseudo-instruction (defined below). |
| C4 | The binary is padded up to the nearest page with at least one <code>hlt</code> instruction (0xf4). |
| C5 | The binary contains no instructions or pseudo-instructions overlapping a 32-byte boundary. |
| C6 | All <i>valid</i> instruction addresses are reachable by a fall-through disassembly that starts at the load (base) address. |
| C7 | All direct control transfers target valid instructions. |

What do these obligations guarantee?

Guarantees

- Data integrity: no loads or stores outside of sandbox
 - Think back to SFI paper
- Reliable disassembly
- No unsafe instructions
- Control flow integrity

NACL Module At Runtime



Transfer from
trusted to
untrusted code,
and vice-versa

Performance - Quake

Run #	Native Client	Linux Executable
1	143.2	142.9
2	143.6	143.4
3	144.2	143.5
Average	143.7	143.3

Table 8: Quake performance comparison. Numbers are in frames per second.